

UNIVERSITY OF THE PEOPLE

CS 2203-01 - AY2026-T4 Programming

Assignment Activity Unit 3

Submitted by: Mubashir Ali

Instructor: Oluwatobi Adedamola Ayilara Adewale

Date: April 26, 2026

Part A: Definition of Normalization and Its Importance

Normalization is a systematic process in database management systems (DBMS) used to organize relational data into well-structured tables that minimize redundancy and dependency anomalies. First introduced by Edgar F. Codd in 1970 alongside the relational model, normalization applies a series of rules known as normal formseach building on the previous to ensure that data is stored logically, consistently, and efficiently (Elmasri & Navathe, 2015).

Formally, normalization transforms an unnormalized or partially normalized relation into a set of smaller, logically coherent relations by identifying and eliminating undesirable functional dependencies. A functional dependency (FD) exists when the value of one attribute (or a set of attributes) uniquely determines the value of another attribute within the same relation

(Ramakrishnan & Gehrke, 2003). Normalization uses these dependencies as the analytical foundation for decomposition.

The importance of normalization in database design cannot be overstated. First, it eliminates data redundancy by the unnecessary repetition of data across rows which wastes storage and increases the risk of inconsistency. Second, normalization prevents three major types of anomalies: (1) insertion anomalies, where new data cannot be added without unrelated data; (2) update anomalies, where changing one record requires multiple updates across many rows; and (3) deletion anomalies, where removing a record inadvertently destroys unrelated information (Connolly & Begg, 2014). Third, a normalized schema enforces data integrity by ensuring each piece of information is stored in exactly one place. Finally, normalized databases are more maintainable and scalable, accommodating future changes without structural overhaul. While normalization may sometimes require additional JOIN operations during querying, the gains in data quality and consistency make it a foundational practice in professional database engineering.

Part B: Normalization Steps - 1NF through BCNF

The original unnormalized relation is:

```
Books (Book_ID, Title, Author, Genre, Publisher, Publication_Year,
                                ISBN, Price)
```

The following assumptions are made based on real-world library knowledge and are used to identify functional dependencies throughout the normalization process:

- Book_ID uniquely identifies each book record.
- ISBN also uniquely identifies a book (it is a candidate key).
- A book may have multiple authors (multi-valued attribute in the raw data).

- Publisher determines Publication_Year for a given book (publisher info is tied to the edition).
- ISBN → Title, Genre, Publisher, Publication_Year, Price (ISBN functionally determines these attributes).
- Publisher → Publication_Year (a transitive dependency through Publisher).

Step 1: First Normal Form (1NF)

A relation is in First Normal Form if all attributes contain only atomic (indivisible) values and there are no repeating groups or multi-valued attributes. In the original relation, the Author attribute may contain multiple author names (e.g., "Alice Smith, Bob Jones"), which violates atomicity.

To achieve 1NF, the Author attribute is decomposed so that each row contains exactly one author. A composite primary key (Book_ID, Author) is introduced to uniquely identify each row.

Book_ID	Title	Author	Genre	Publisher	Pub Year	ISBN	Price
B001	Database Design	Alice Smith	Technology	Pearson	2020	978-0-13-1	$59.99
B001	Database Design	Bob Jones	Technology	Pearson	2020	978-0-13-1	$59.99
B002	Data Structures	Carol Lee	Technology	O'Reilly	2019	978-1-49-2	$49.99

Table 1: Books relation in 1NF - Primary Key: (Book_ID, Author)

Step 2: Second Normal Form (2NF)

A relation is in Second Normal Form if it is in 1NF and every non-key attribute is fully functionally dependent on the entire composite primary key not just a part of it. This issue is called a partial dependency.

In the 1NF table above, the composite key is (Book_ID, Author). However, attributes such as Title, Genre, Publisher, Publication_Year, ISBN, and Price depend solely on Book_ID, not on Author. This is a partial dependency and must be eliminated by decomposing the relation into two tables.

Book_ID (PK)	Title	Genre	Publisher	Pub_Year	ISBN	Price
B001	Database Design	Technology	Pearson	2020	978-0-13-1	$59.99
B002	Data Structures	Technology	O'Reilly	2019	978-1-49-2	$49.99

Table 2a: Books table in 2NF

Book_ID (PK)	Author (PK)
B001	Alice Smith
B001	Bob Jones
B002	Carol Lee

Table 2b: BookAuthors table in 2NF Composite PK: (Book_ID, Author)

Step 3: Third Normal Form (3NF)

A relation is in Third Normal Form if it is in 2NF and contains no transitive dependencies that is, no non-key attribute depends on another non-key attribute. In the Books table (2NF), the following transitive dependency exists:

`Book_ID → Publisher → Publication_Year`

Publication_Year is determined by Publisher, not directly by Book_ID. This is a transitive dependency. To achieve 3NF, Publisher and Publication_Year are extracted into a separate Publishers table.

Book_ID (PK)	Title	Genre	Publisher_ID (FK)	ISBN	Price
B001	Database Design	Technology	PUB01	978-0-13-1	$59.99
B002	Data Structures	Technology	PUB02	978-1-49-2	$49.99

Table 3a: Books table in 3NF

Publisher_ID (PK)	Publisher_Name	Publication_Year
PUB01	Pearson	2020
PUB02	O'Reilly	2019

Table 3b: Publishers table in 3NF

Step 4: Boyce-Codd Normal Form (BCNF)

A relation is in Boyce-Codd Normal Form if, for every non-trivial functional dependency X → Y, X is a superkey of the relation. BCNF is stricter than 3NF because 3NF permits certain dependencies where the right-hand side is part of a candidate key, while BCNF does not (Ramakrishnan & Gehrke, 2003).

Examining the 3NF tables: in the Books table, both Book_ID and ISBN are candidate keys, and all non-trivial functional dependencies have a superkey as their determinant. In the Publishers table, Publisher_ID is the only candidate key and all dependencies satisfy BCNF. The BookAuthors table uses a composite key with no non-key attributes, so it trivially satisfies BCNF. Therefore, the 3NF decomposition already satisfies BCNF for this schema. The final BCNF schema is:

Relation	Attributes	Primary Key	Normal Form
Books	Book_ID, Title, Genre, Publisher_ID (FK), ISBN, Price	Book_ID	BCNF
Publishers	Publisher_ID, Publisher_Name, Publication_Year	Publisher_ID	BCNF
BookAuthors	Book_ID (FK), Author	(Book_ID, Author)	BCNF

Table 4: Final BCNF schema summary

Part C: Functional Dependencies and How Normalization Addresses Them

The following functional dependencies were identified in the original unnormalized relation and guided each normalization step:

Functional Dependency	Type	Normal Form Applied	Resolution
Book_ID → Title, Genre, Publisher, Pub_Year, ISBN, Price	Full FD on PK	1NF → 2NF	Retained in Books table; Author separated
(Book_ID, Author) → [all attributes]	Partial dependency on composite key	2NF	Decomposed into Books and BookAuthors tables
Book_ID → Publisher → Publication_Year	Transitive dependency	3NF	Publisher extracted to Publishers table
ISBN → Title, Genre, Publisher, Price	Candidate key FD	BCNF	ISBN recognized as alternate key; schema already in BCNF
Publisher_ID → Publisher_Name, Publication_Year	Full FD on PK	BCNF	Satisfied; Publisher_ID is a superkey

The most consequential dependency was the partial dependency of Title, Genre, Publisher, Publication_Year, ISBN, and Price on only Book_ID within the composite key (Book_ID, Author). This caused significant redundancy: all book details were repeated for every author. The 2NF decomposition resolved this by separating book metadata from authorship data.

The transitive dependency Publisher → Publication_Year introduced an update anomaly: changing a publisher's year would require updating every book record associated with that publisher. The 3NF step resolved this by isolating publisher data in its own table, ensuring each fact is stored exactly once. As Elmasri and Navathe (2015) note, eliminating transitive dependencies is essential for maintaining data integrity without requiring cascading updates.

Part D: Advantages and Drawbacks of Higher Normal Forms

Advantages of 3NF and BCNF

Achieving higher normal forms yields significant benefits in database design and data integrity. First, by eliminating redundancy, 3NF and BCNF drastically reduce storage requirements and prevent the propagation of inconsistent data. Second, update anomalies are eliminated: because each fact is stored in exactly one place, a single update operation maintains consistency across the entire database (Connolly & Begg, 2014). Third, the schema becomes more modular and maintainable; adding a new publisher, for instance, requires inserting only one row in the Publishers table rather than updating every associated book record.

From a data integrity perspective, BCNF provides the strongest guarantee: every determinant in every non-trivial functional dependency is a superkey, meaning no spurious dependencies can introduce anomalies. This is particularly valuable in systems where data correctness is critical, such as financial records, healthcare databases, or legal document management.

Drawbacks and Trade-offs

Despite these advantages, higher normal forms introduce notable trade-offs. The primary concern is query performance. Decomposing a relation into multiple tables requires JOIN operations to reconstruct the original data during queries. In large-scale systems with millions of records, multi-table JOINs can be computationally expensive and may slow query response times significantly (Ramakrishnan & Gehrke, 2003).

Furthermore, BCNF decomposition does not always preserve all functional dependencies. In cases where a dependency spans the decomposed tables, enforcing it requires application-level logic or complex constraints, increasing development complexity. As Date (2003) points out,

dependency preservation is a critical practical consideration that sometimes makes 3NF preferable to BCNF in production systems.

In contrast, lower normal forms (1NF and 2NF) offer simpler schemas and faster reads by keeping related data together, at the cost of redundancy and anomaly risk. This trade-off has led many real-world systems to adopt a hybrid approach: designing to 3NF for transactional databases (OLTP) while intentionally denormalizing for analytical workloads (OLAP) in data warehouses, where read performance dominates (Kimball & Ross, 2013).

In conclusion, the optimal normal form depends on the specific requirements of the system: 3NF and BCNF are ideal for transactional integrity, while controlled denormalization may be appropriate for performance-critical analytical systems.

References

Connolly, T., & Begg, C. (2014). Database systems: A practical approach to design, implementation, and management (6th ed.). Pearson Education.

Date, C. J. (2003). An introduction to database systems (8th ed.). Addison-Wesley.

Elmasri, R., & Navathe, S. B. (2015). Fundamentals of database systems (7th ed.). Pearson Education.

Kimball, R., & Ross, M. (2013). The data warehouse toolkit: The definitive guide to dimensional modeling (3rd ed.). Wiley.

Ramakrishnan, R., & Gehrke, J. (2003). Database management systems (3rd ed.). McGraw-Hill Education.